Virtual Memory

Chapter 8

Basic Idea of Virtual Memory Management

- A process may be swapped in and out of main memory such that it occupies different regions
- Memory references in logical addresses are dynamically translated into physical addresses at run time
- A process may be broken up into pieces that do not need to located contiguously(连续的) in main memory
- All pieces of a process do not need to be loaded in main memory during execution

Execution of a Program

- Operating system only brings into main memory a few pieces of the program
- An exception(Page Fault, 页面故障) is generated when an address is needed that is not in main memory. Operating system places the process in blocking state
- Piece of process that contains the logical address is brought into main memory
  - Operating system issues a disk I/O Read request
  - Another process is dispatched to run while the disk I/O takes place
  - An interrupt is issued when disk I/O complete which causes the operating system to place the affected process in the Ready state

Advantages of Breaking up a Process

- More processes may be maintained in main memory
  - Only load in some of the pieces of each process
  - With so many processes in main memory, it is very likely a process will be in the Ready state at any particular time
- A process may be larger than all of main memory

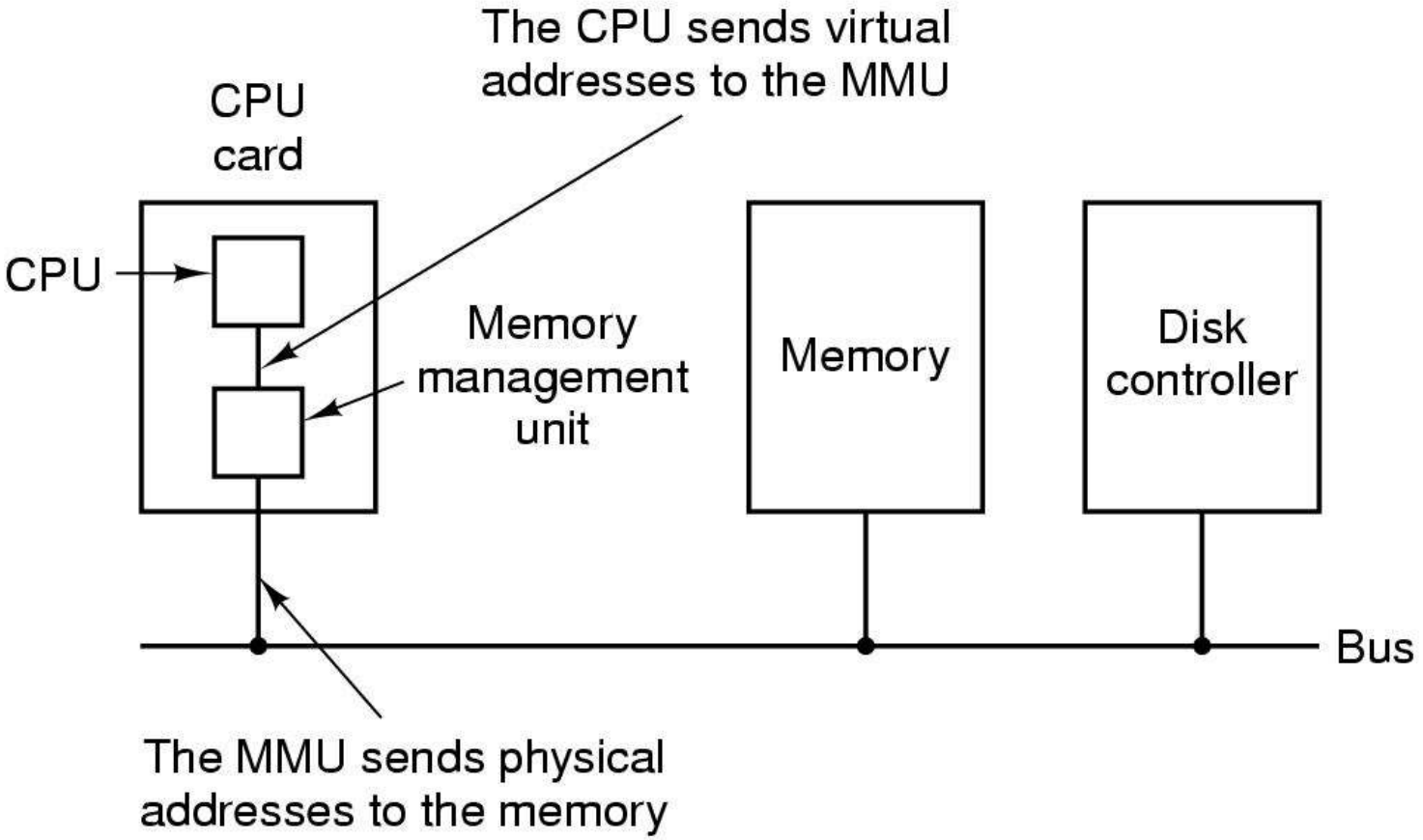
Principle of Locality(局部性原理)

- Program(code/text) and data references within a process tend to cluster
- Only a few pieces of a process will be needed over a short period of time
- Possible to make intelligent guesses about which pieces will be needed in the future, this suggests that virtual memory may work efficiently
- Resident set(驻留集) - portion of process that is in main memory

Support Needed for Virtual Memory

- Hardware must support paging and segmentation
- Operating system must be able to manage the movement of pages and/or segments between secondary memory and main memory

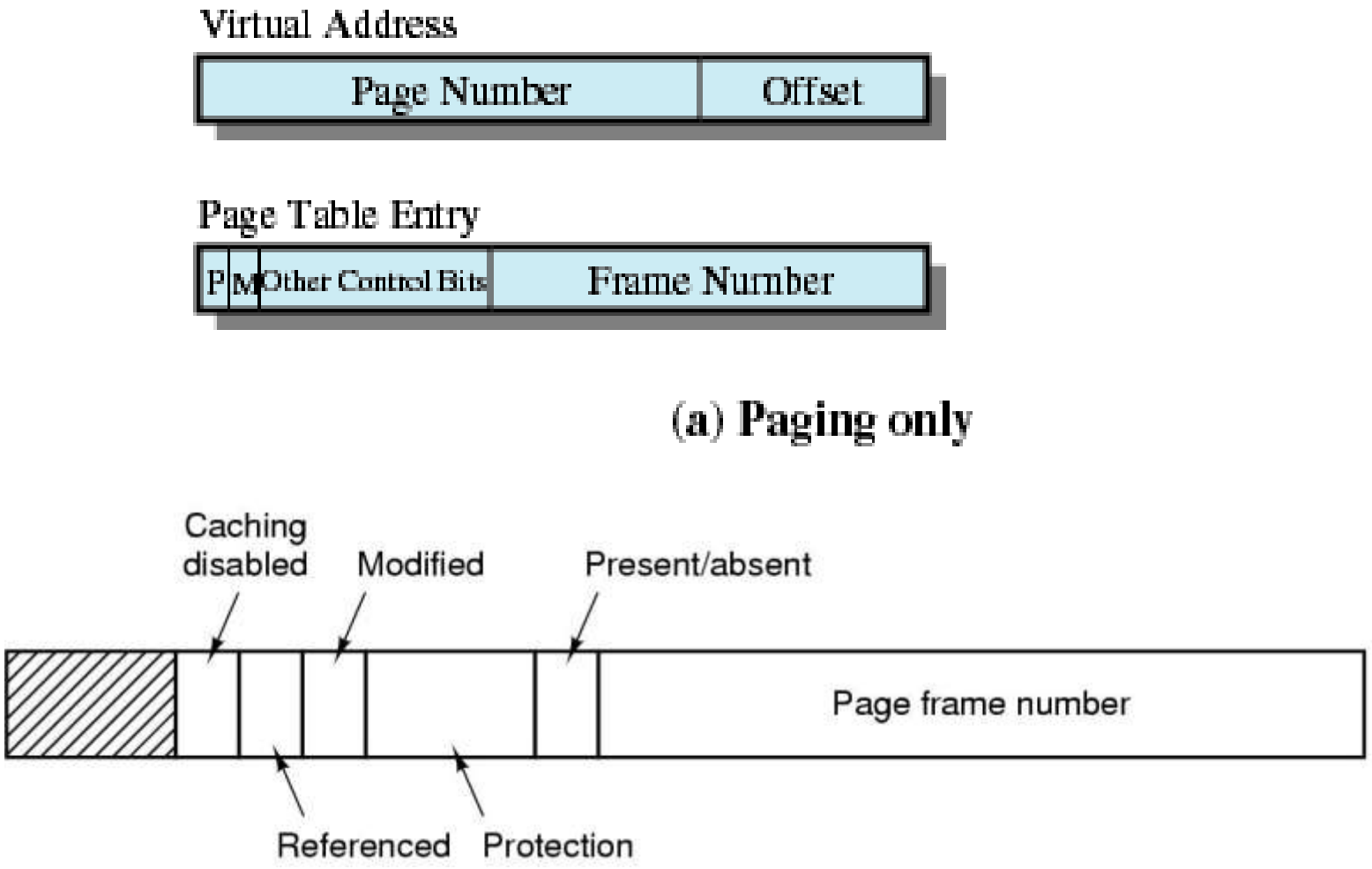
Paging


The CPU sends virtual addresses to the MMU
CPU card
CPU
Memory management unit
Memory
Disk controller
Bus
The MMU sends physical addresses to the memory

Paging

- Each process has its own page table
- Each page table entry contains the frame number of the corresponding page in main memory.
  - A bit is needed in PTE to indicate whether the page is in main memory or not
- Virtual Address Space

Virtual Address and Page Table Entry



Modify Bit in Page Table

- Modify bit is needed to indicate if the page has been altered since it was last loaded into main memory
- If no change has been made, the page does not have to be written to the disk when it needs to be swapped out

Address Translation(Mapping)

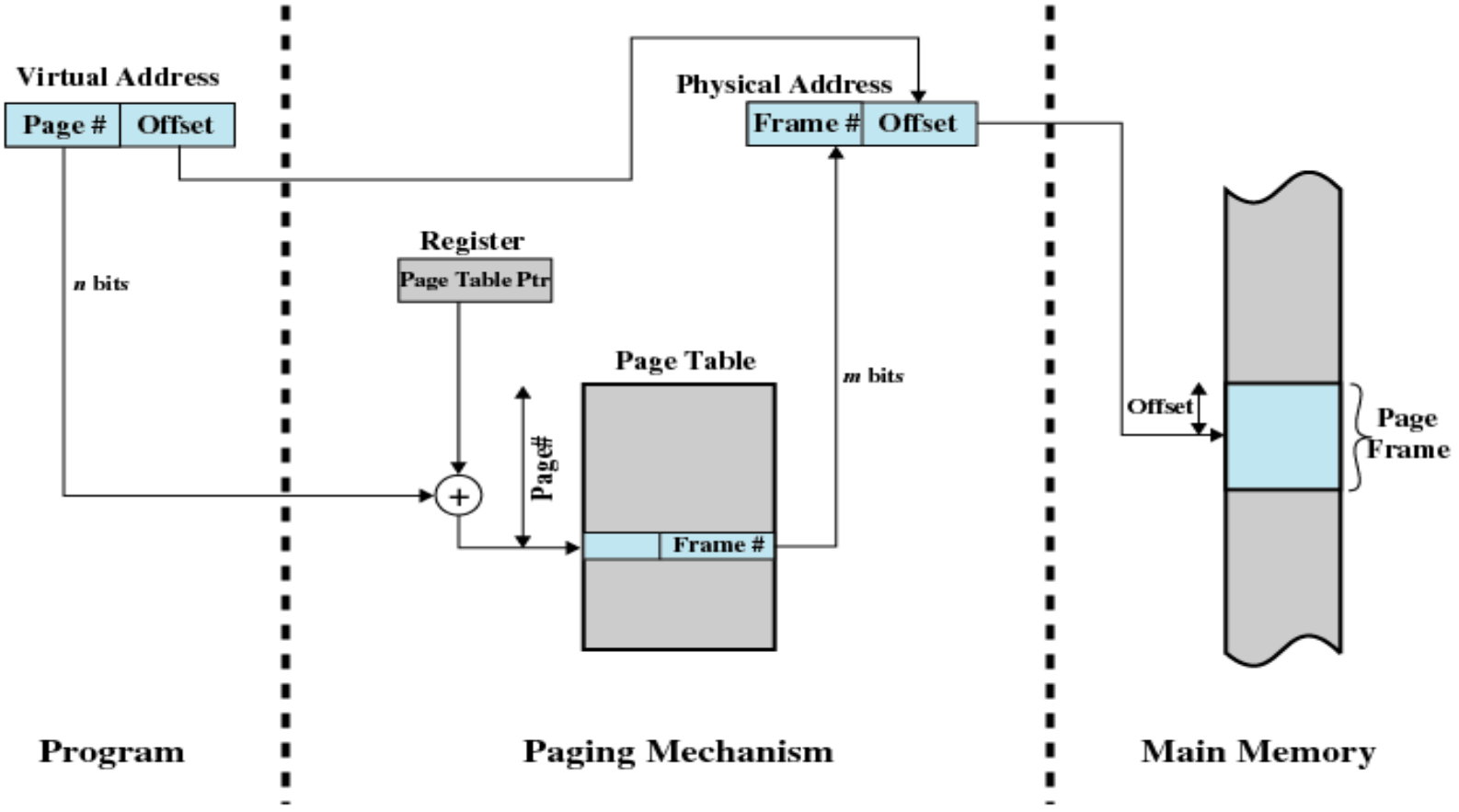
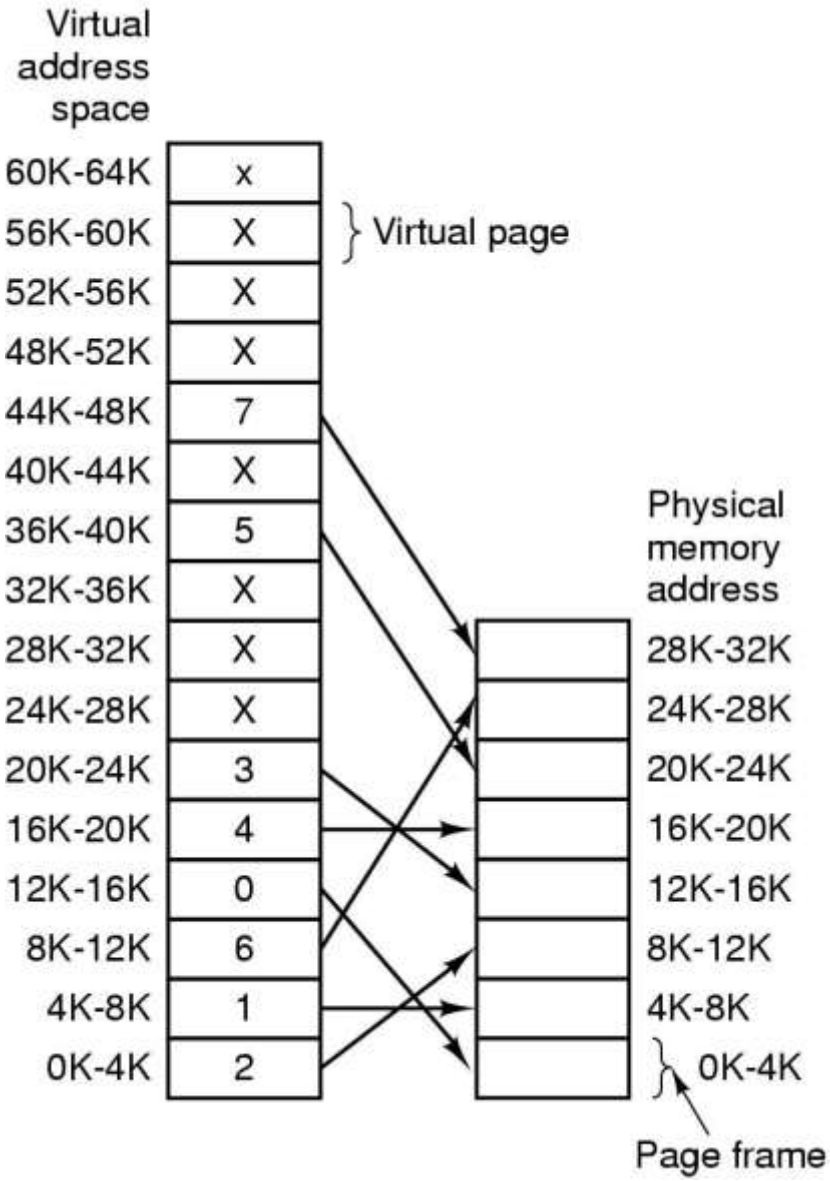
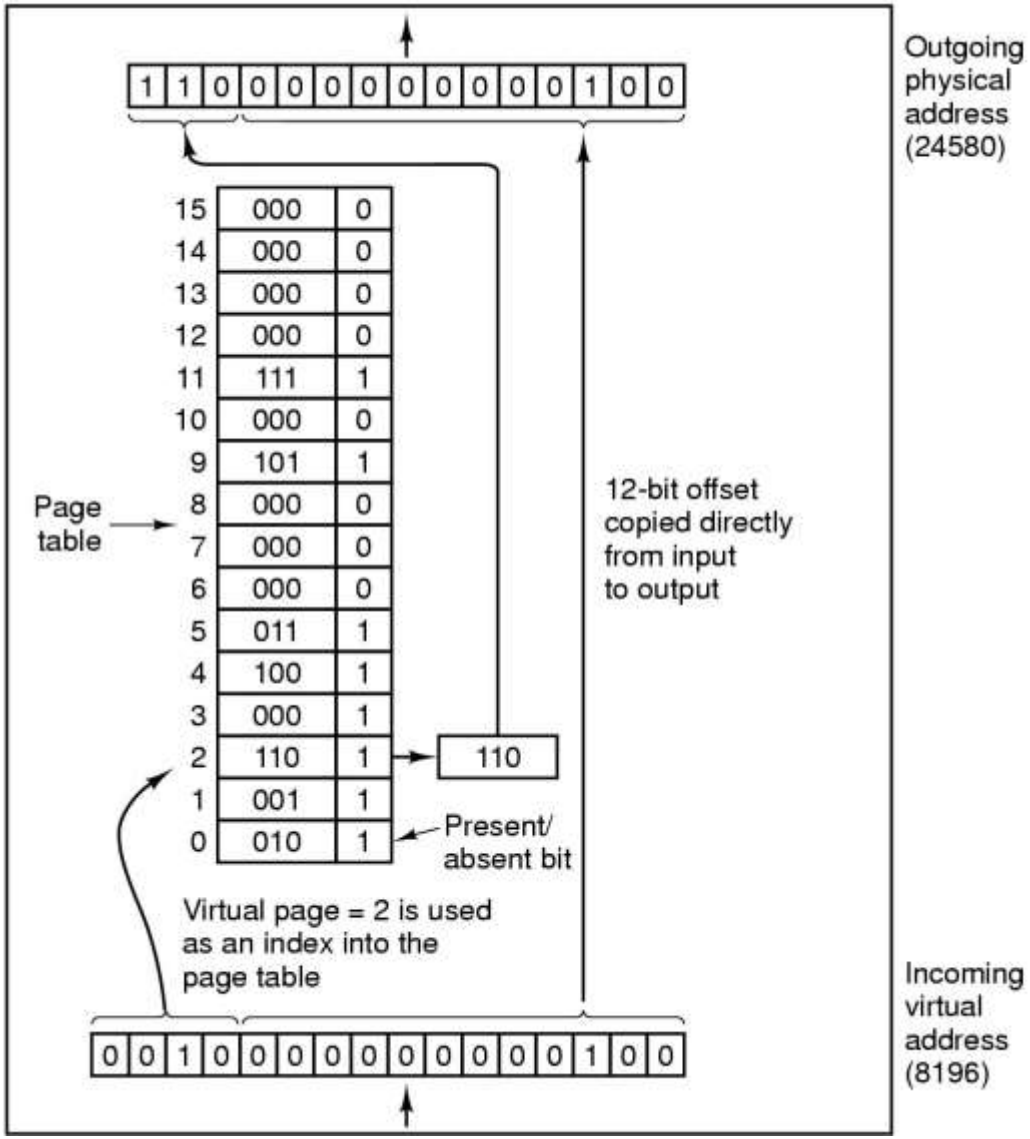


Figure 8.3 Address Translation in a Paging System

Address Translation (Cont.)



Address Translation (Cont.)



Page Tables

- The entire page table may take up too much main memory
- Page tables are also stored in virtual memory
- When a process is running, part of its page table is in main memory

Two-Level Scheme for 32-bit Address

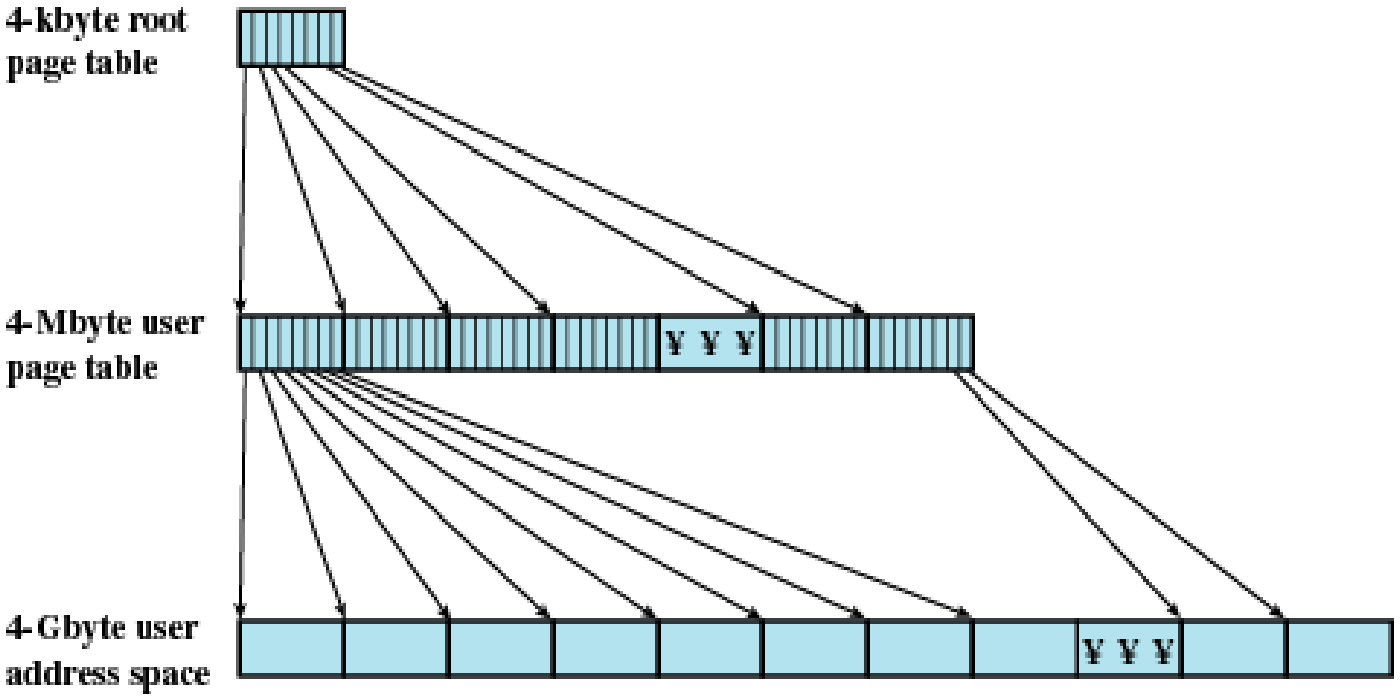
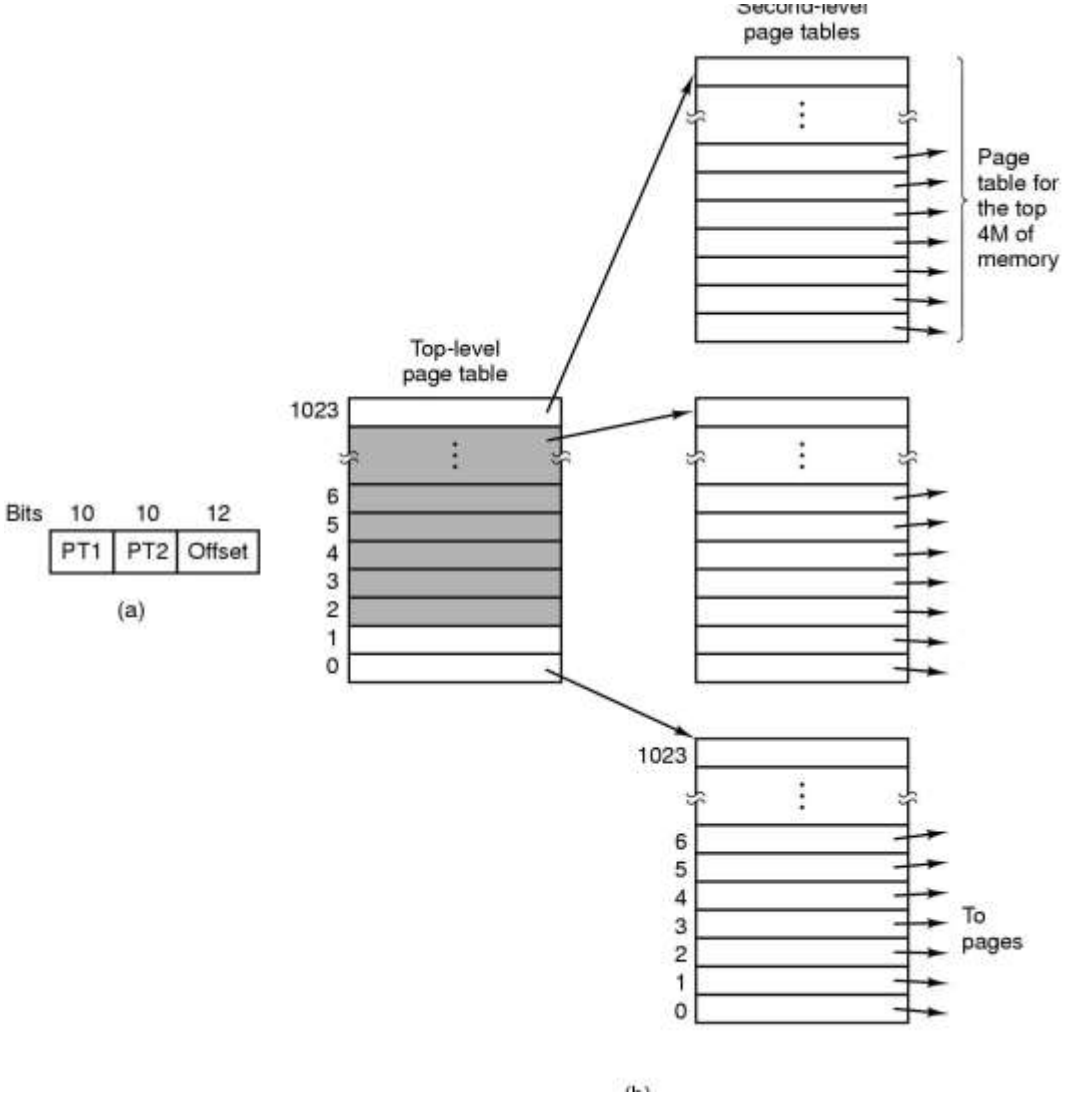
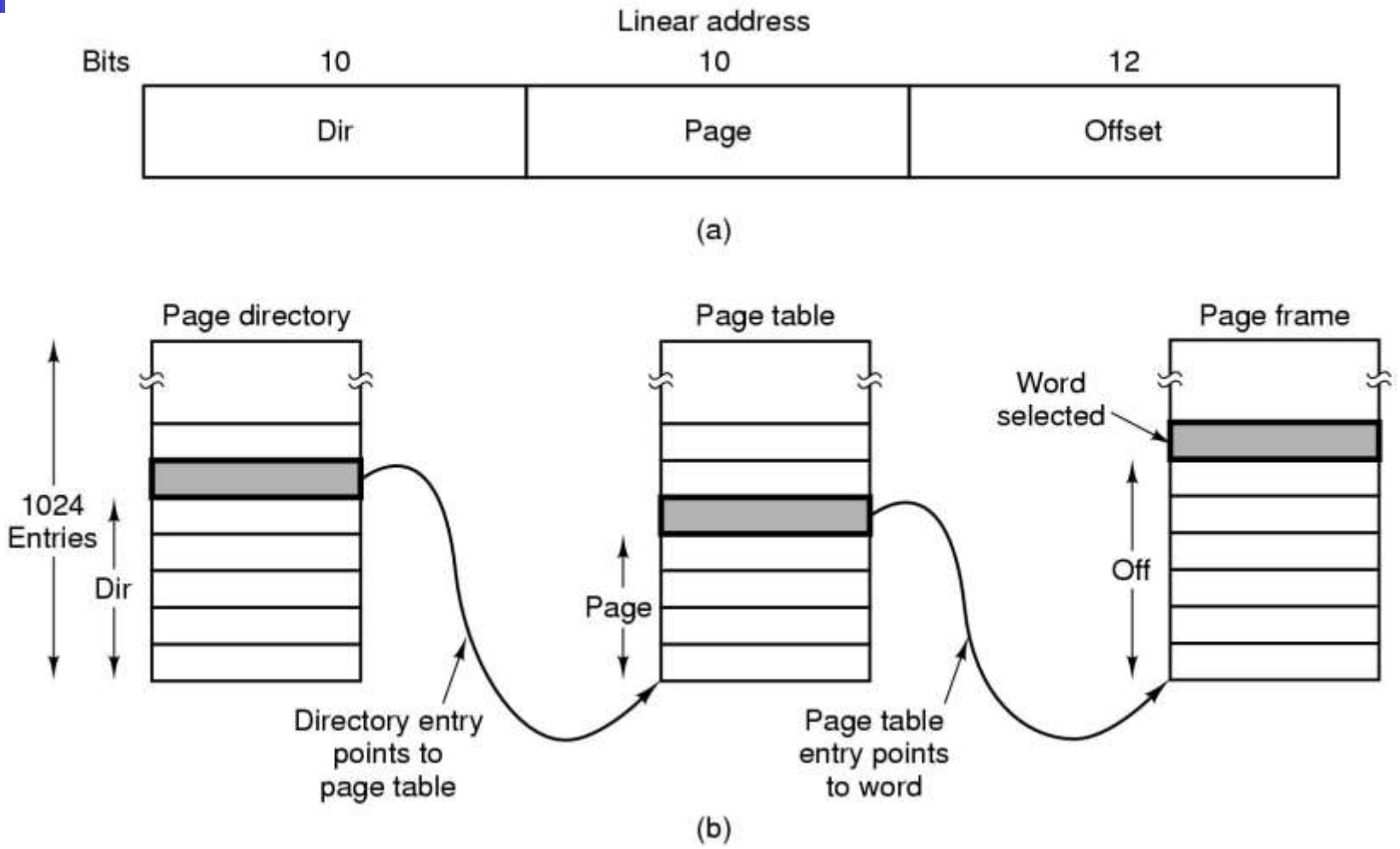


Figure 8.4 A Two-Level Hierarchical Page Table

Two-Level Scheme



Two-Level Scheme



Inverted Page Table(逆向页表)

- Used on PowerPC, UltraSPARC, and IA-64 architecture
- Page number portion of a virtual address is mapped into a hash value
- Hash value points to inverted page table
- Fixed proportion of real memory is required for the tables regardless of the number of processes

Inverted Page Table

- Page number
- Process identifier
- Control bits
- Chain pointer

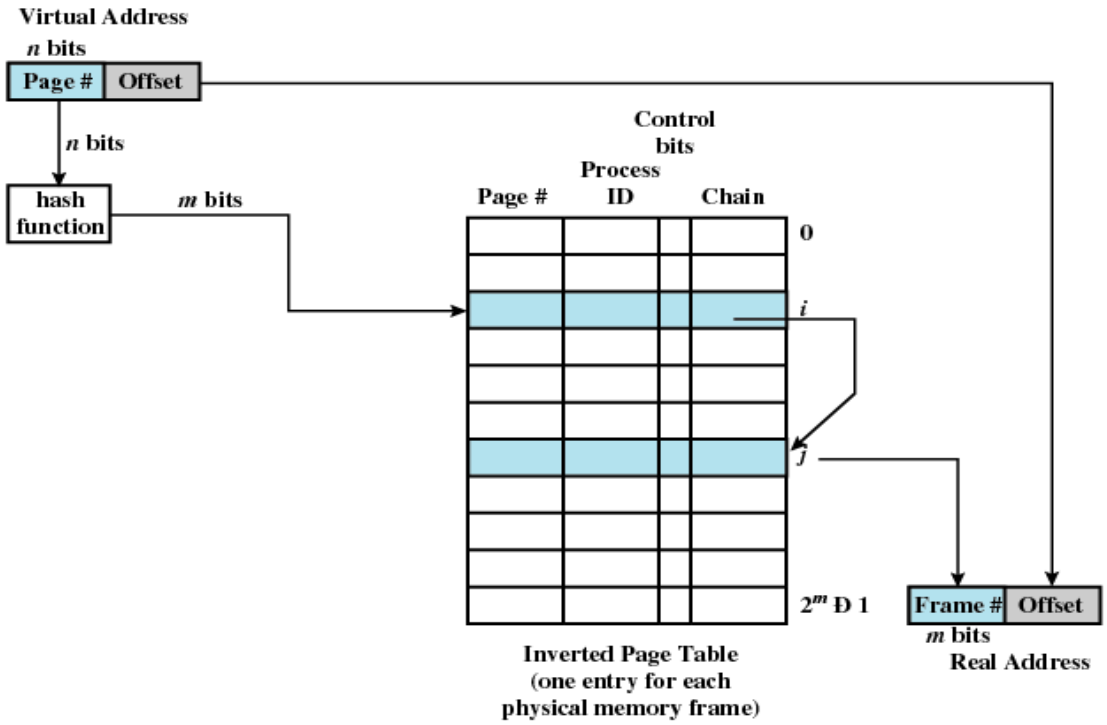
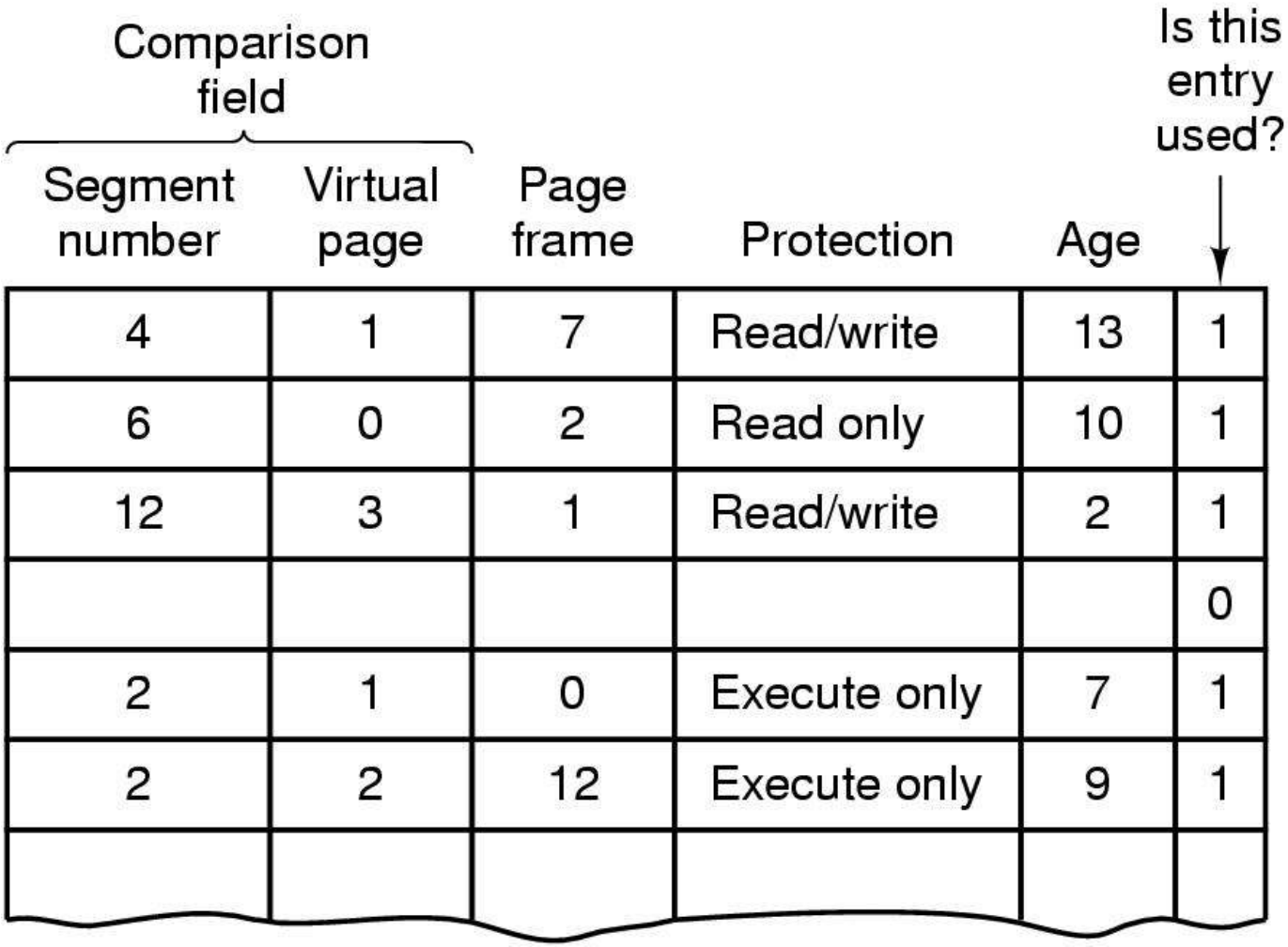


Figure 8.6 Inverted Page Table Structure

Translation Lookaside Buffer

- Each virtual memory reference can cause two physical memory accesses
  - One to fetch the page table
  - One to fetch the data
- To overcome this problem a high-speed cache is set up for page table entries, called a Translation Lookaside Buffer (TLB)
- TLB contains page table entries that have been most recently used

Translation Lookaside Buffer



Comparison field: Segment number	Comparison field: Virtual page	Page frame	Protection	Age	Is this entry used?
4	1	7	Read/write	13	1
6	0	2	Read only	10	1
12	3	1	Read/write	2	1
					0
2	1	0	Execute only	7	1
2	2	12	Execute only	9	1

Translation Lookaside Buffer

- Given a virtual address, processor examines the TLB
- If page table entry is present (TLB hit), the frame number is retrieved and the real address is formed
- If page table entry is not found in the TLB (TLB miss), the page number is used to index the process page table

Translation Lookaside Buffer

- First checks if page is already in main memory
  - If not in main memory a page fault is issued
- The TLB is updated to include the new page entry

Translation Lookaside Buffer

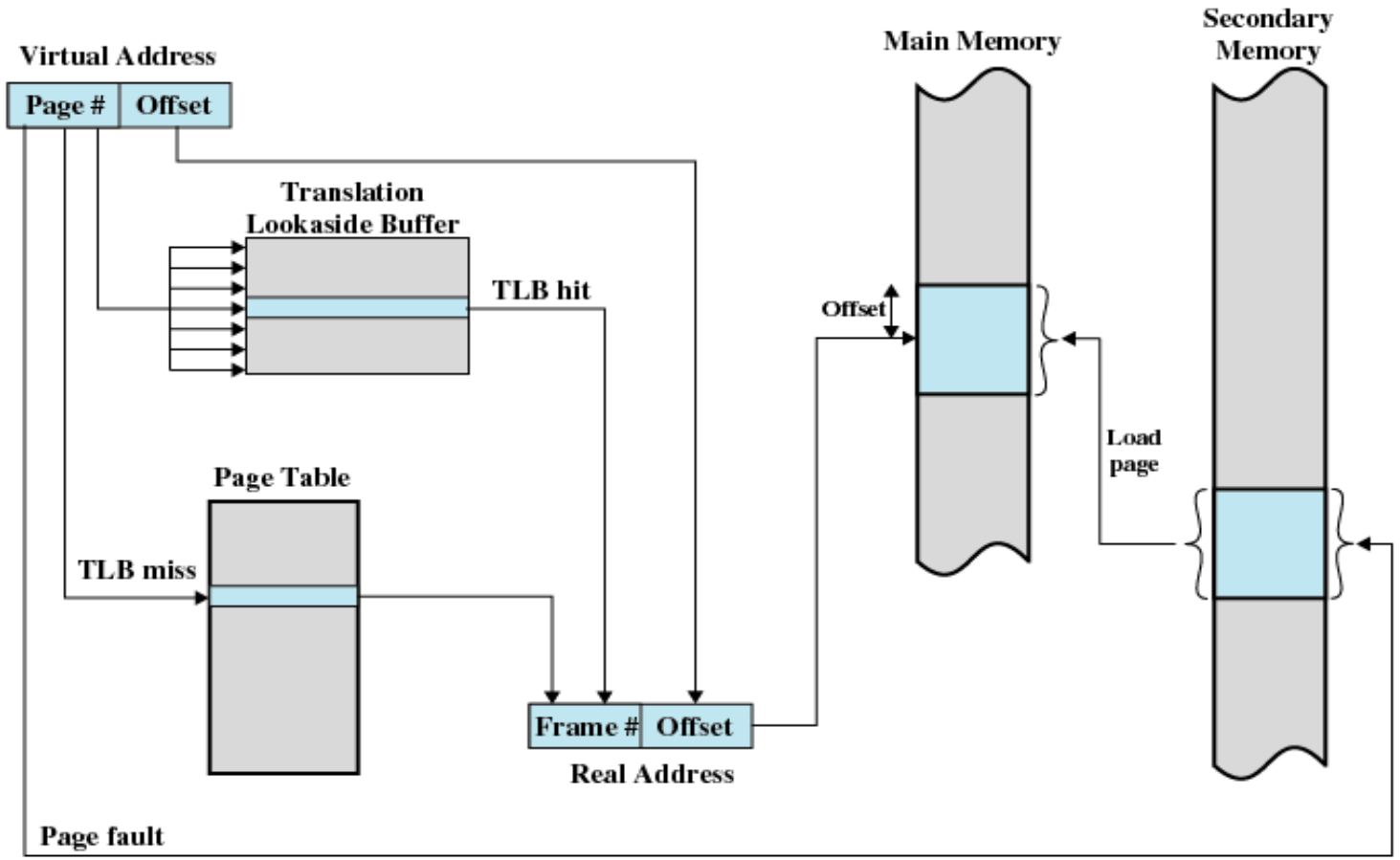


Figure 8.7 Use of a Translation Lookaside Buffer

Translation Lookaside Buffer

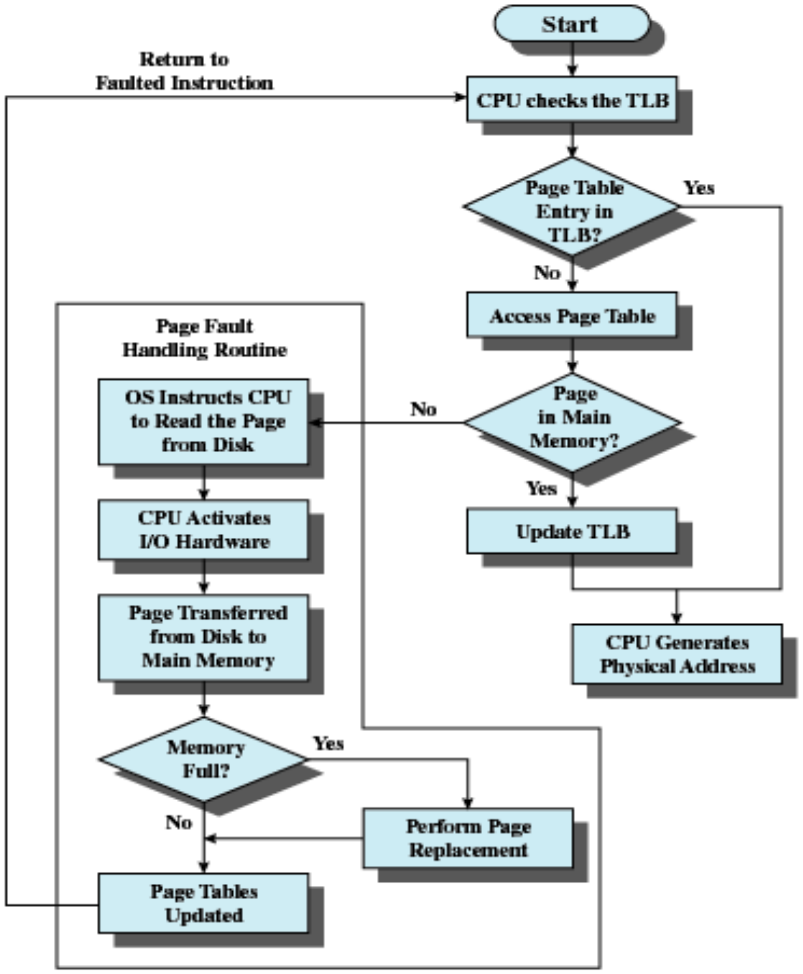


Figure 8.8 Operation of Paging and Translation Lookaside Buffer (TLB) [FURH87]

Translation Lookaside Buffer

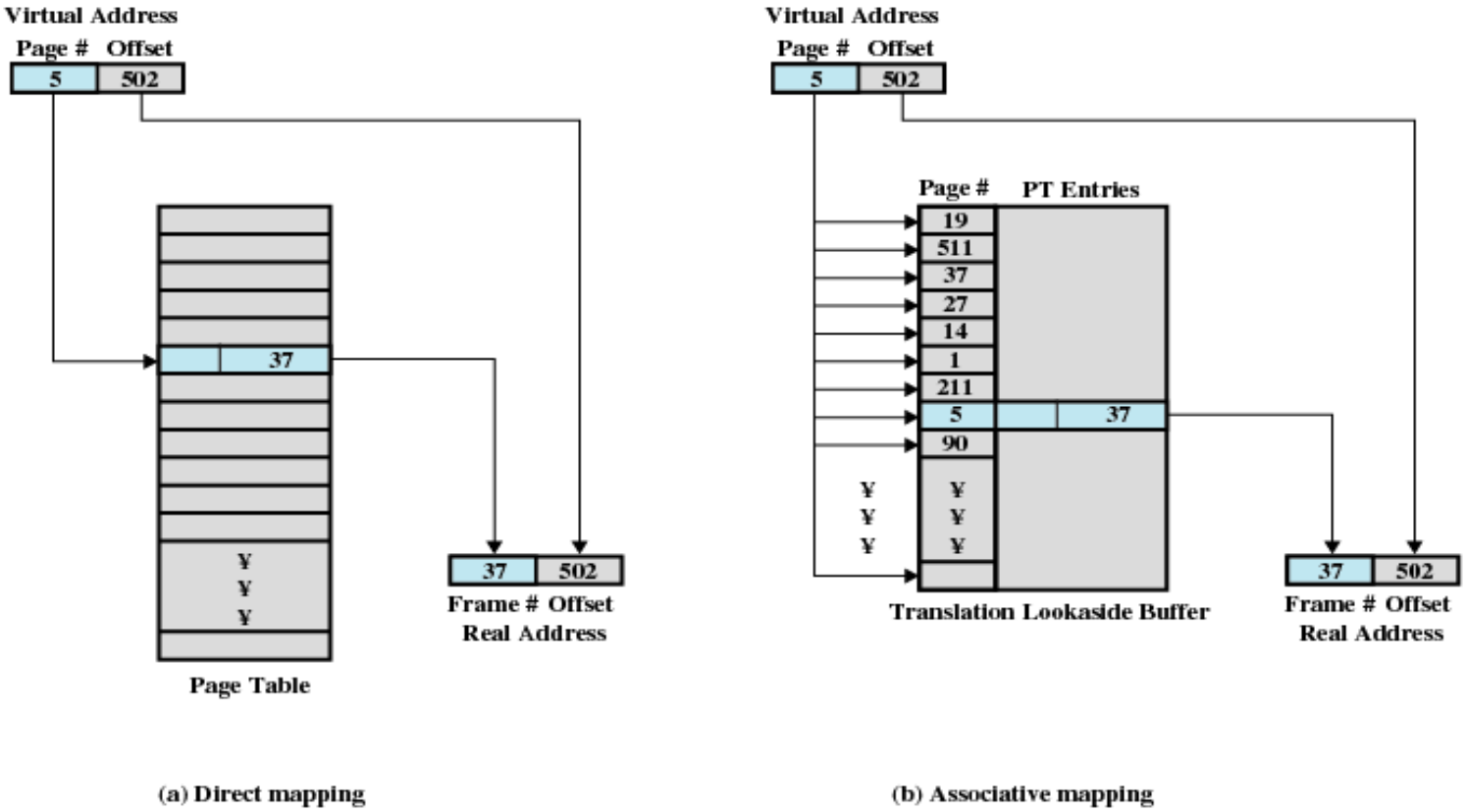


Figure 8.9 Direct Versus Associative Lookup for Page Table Entries

Translation Lookaside Buffer

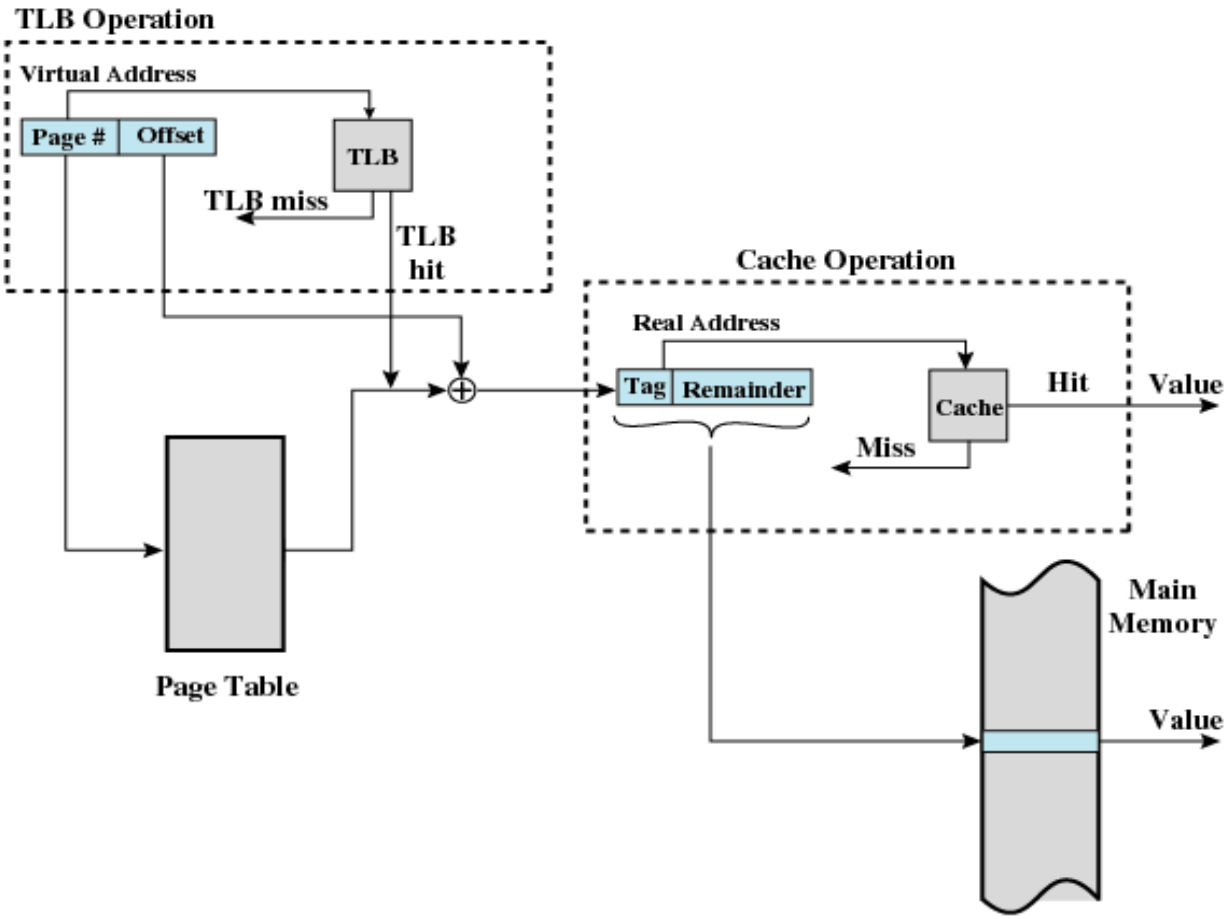
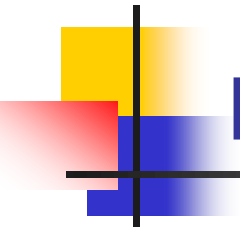


Figure 8.10 Translation Lookaside Buffer and Cache Operation

Page Size

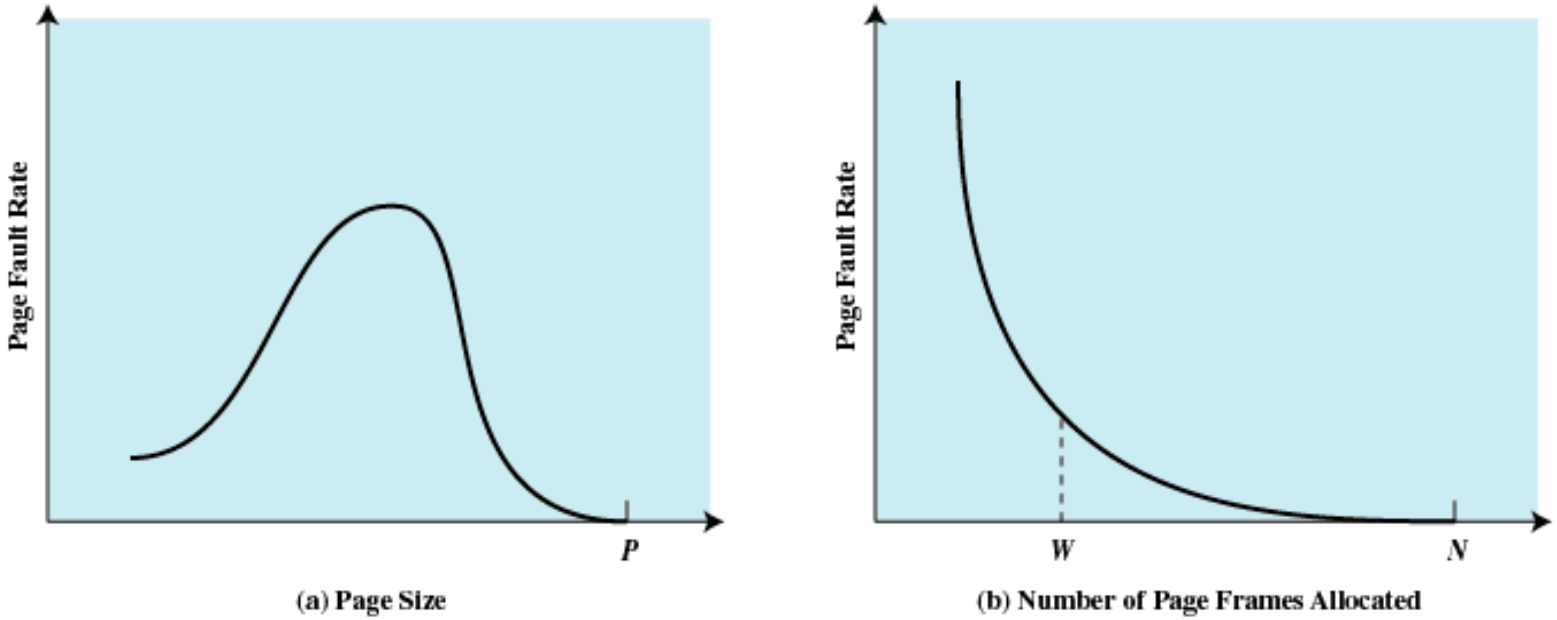
- Smaller page size, less amount of internal fragmentation
- Smaller page size, more pages required per process
- More pages per process means larger page tables
- Larger page tables means large portion of page tables in virtual memory
- Secondary memory is designed to efficiently transfer large blocks of data so a large page size is better



Page Size

- Small page size, large number of pages will be found in main memory
- As time goes on during execution, the pages in memory will all contain portions of the process near recent references. Page faults low.
- Increased page size causes pages to contain locations further from any recent reference. Page faults rise.

Paging Behavior of a Program



P = size of entire process
W = working set size
N = total number of pages in process

Figure 8.11 Typical Paging Behavior of a Program

Example Page Sizes

Table 8.2 Example Page Sizes

Computer	Page Size
Atlas	512 48-bit words
Honeywell-Multics	1024 36-bit word
IBM 370/XA and 370/ESA	4 Kbytes
VAX family	512 bytes
IBM AS/400	512 bytes
DEC Alpha	8 Kbytes
MIPS	4 kbyes to 16 Mbytes
UltraSPARC	8 Kbytes to 4 Mbytes
Pentium	4 Kbytes or 4 Mbytes
PowerPc	4 Kbytes
Itanium	4 Kbytes to 256 Mbytes

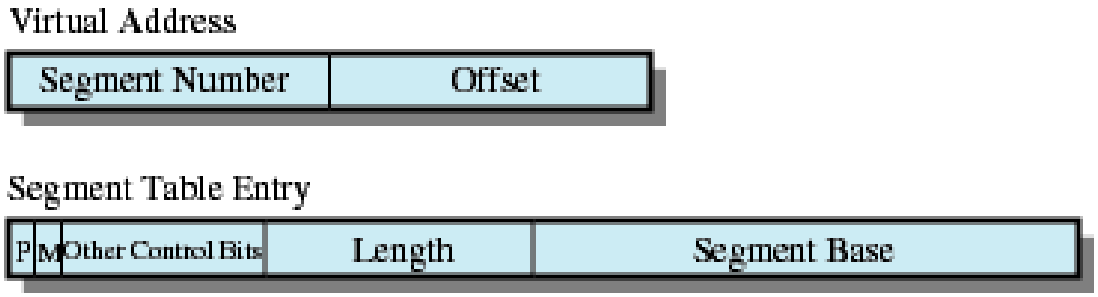
Segmentation(分段)

- May be unequal, dynamic size
- Simplifies handling of growing data structures
- Allows programs to be altered and recompiled independently
- Lends itself to sharing data among processes
- Lends itself to protection

Segment Tables

- Corresponding segment in main memory
- Each entry contains the length of the segment
- A bit is needed to determine if segment is already in main memory
- Another bit is needed to determine if the segment has been modified since it was loaded in main memory

Segment Table Entries



(b) Segmentation only

Address Translation in Segmentation System

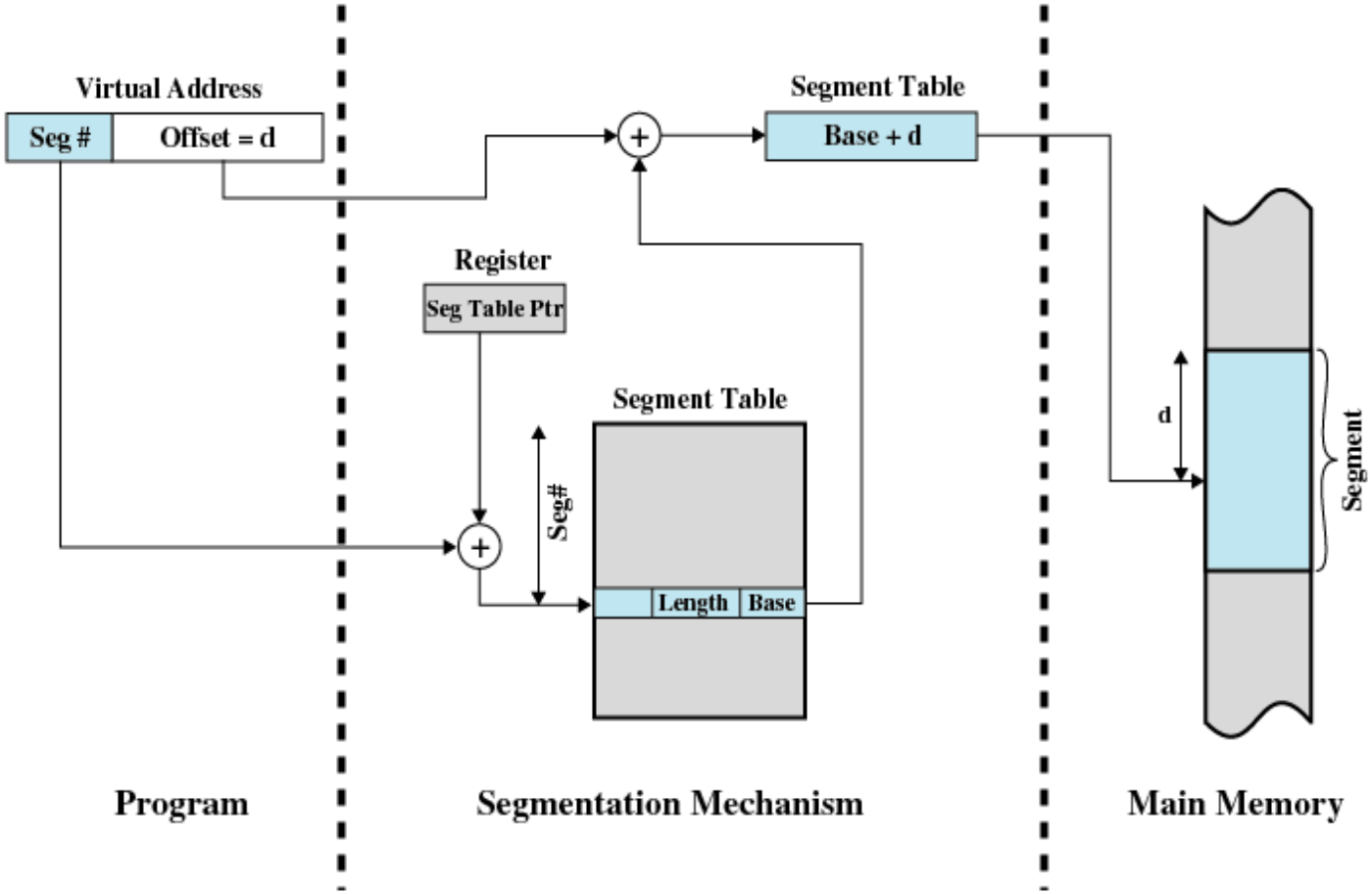
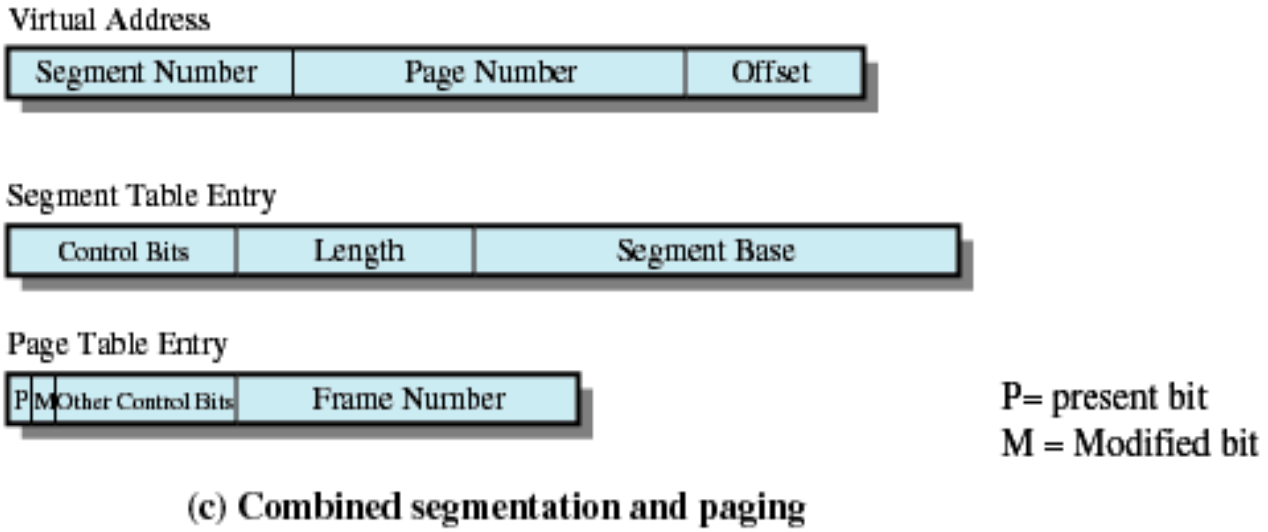


Figure 8.12 Address Translation in a Segmentation System

Combined Paging and Segmentation

- Paging is transparent(透明) to the programmer
- Segmentation is visible to the programmer
- Each segment is broken into fixed-size pages



(c) Combined segmentation and paging

Combined Paging and Segmentation

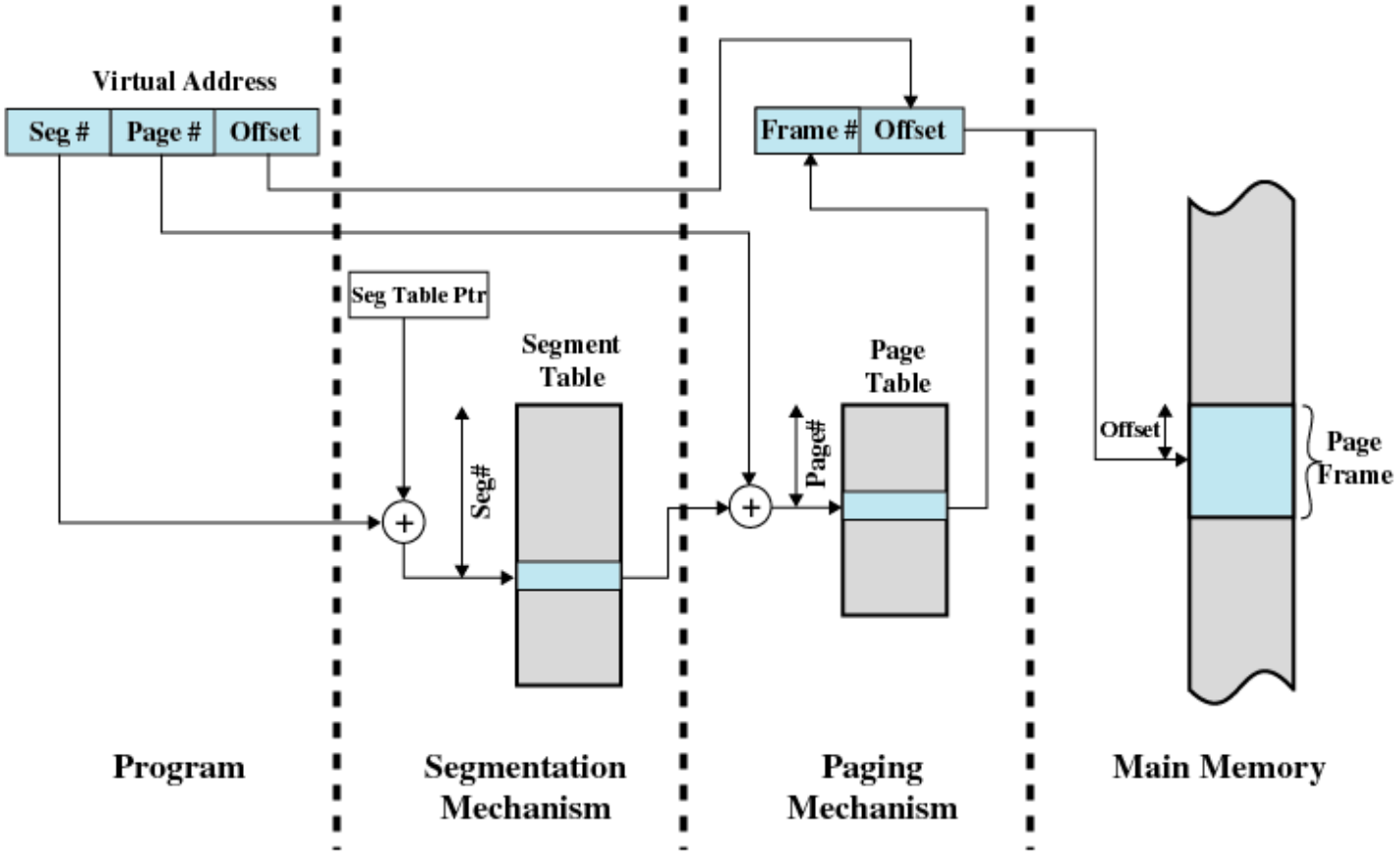


Figure 8.13 Address Translation in a Segmentation/Paging System

Protection Relationship Between Segments

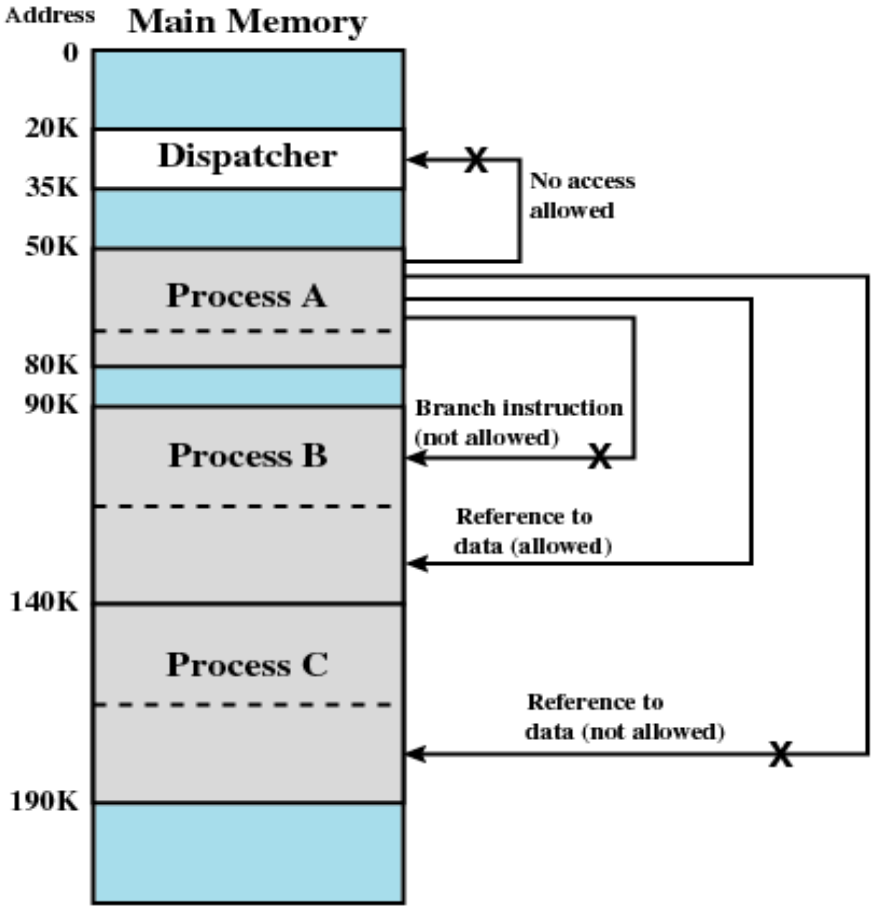


Figure 8.14 Protection Relationships Between Segments

Fetch Policy(页面何时调入)

- Determines when a page should be brought into memory
- Demand paging only brings pages into main memory when a reference is made to a location on the page
  - Many page faults when process first started
- Prepaging(预取) brings in more pages than needed
  - More efficient to bring in pages that reside contiguously on the disk

Replacement Policy(替换策略)

- Which page is replaced?
- Page removed should be the page least likely to be referenced in the near future
- Most policies predict the future behavior on the basis of past behavior
- Thrashing(颠簸)
  - Swapping out a piece of a process just before that piece is needed
  - The processor spends most of its time swapping pieces rather than executing user instructions

Frame Locking

- Some of the frames in main memory may be locked. When a frame is locked, it may not be replaced
  - Kernel of the operating system
  - Key control structures
  - I/O buffers
- Locking is achieved by associating a lock bit with each frame

Basic Replacement Algorithms

- Optimal policy
- Least Recently Used (LRU)
- First-in, first-out (FIFO)
- Clock Policy

Optimal Policy

- Selects for replacement that page for which the time to the next reference is the longest
- Impossible to have perfect knowledge of future events

Least Recently Used(最久未使用)

- Replaces the page that has not been referenced for the longest time
- By the principle of locality, this should be the page least likely to be referenced in the near future
- Each page could be tagged with the time of last reference. This would require a great deal of overhead

First-In First-Out (FIFO)

- Treats page frames allocated to a process as a circular buffer
- Pages are removed in round-robin style
- Simplest replacement policy to implement
- Page that has been in memory the longest is replaced

Clock Policy

- Additional bit called a use bit
- When a page is first loaded in memory, the use bit is set to 1
- When the page is referenced, the use bit is set to 1
- When it is time to replace a page, the first frame encountered with the use bit set to 0 is replaced.
- During the search for replacement, each use bit set to 1 is changed to 0

Four Page-Replacement Algorithms

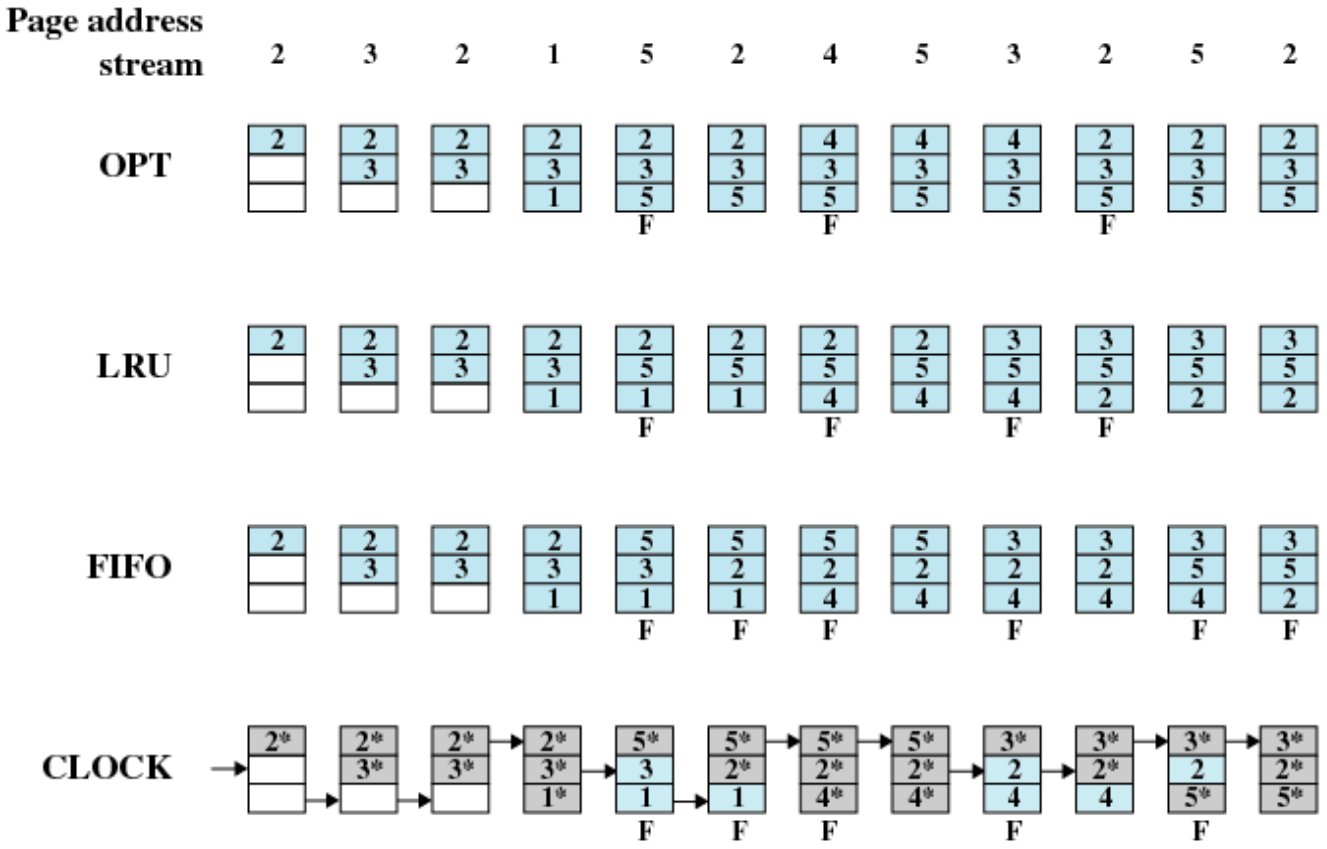
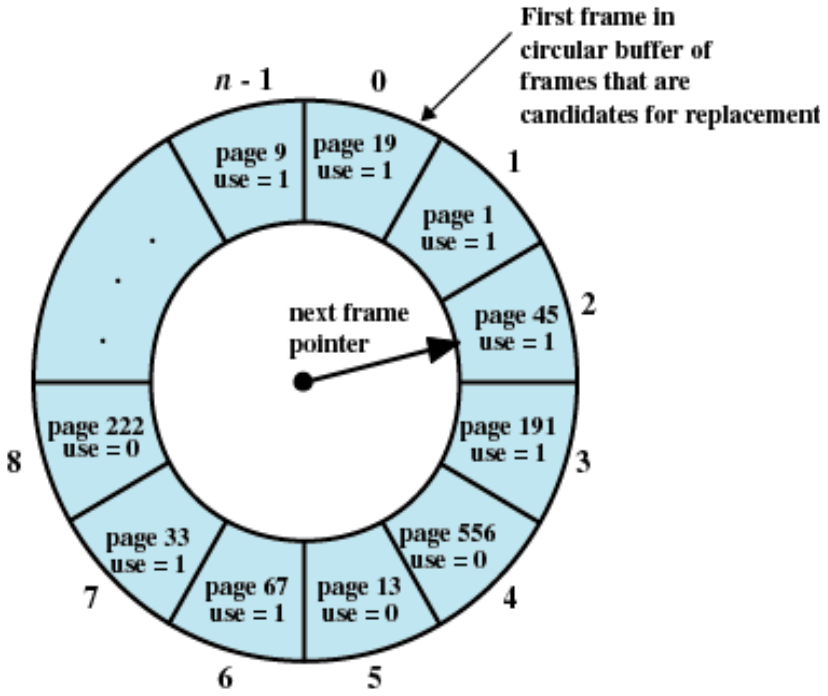
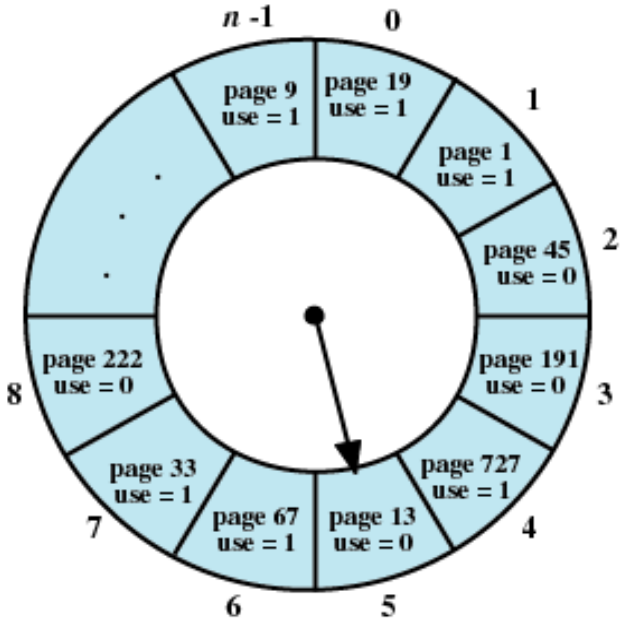


Figure 8.15 Behavior of Four Page-Replacement Algorithms

Example of Clock Policy



(a) State of buffer just prior to a page replacement



(b) State of buffer just after the next page replacement

Figure 8.16 Example of Clock Policy Operation

Comparison of Placement Algorithms

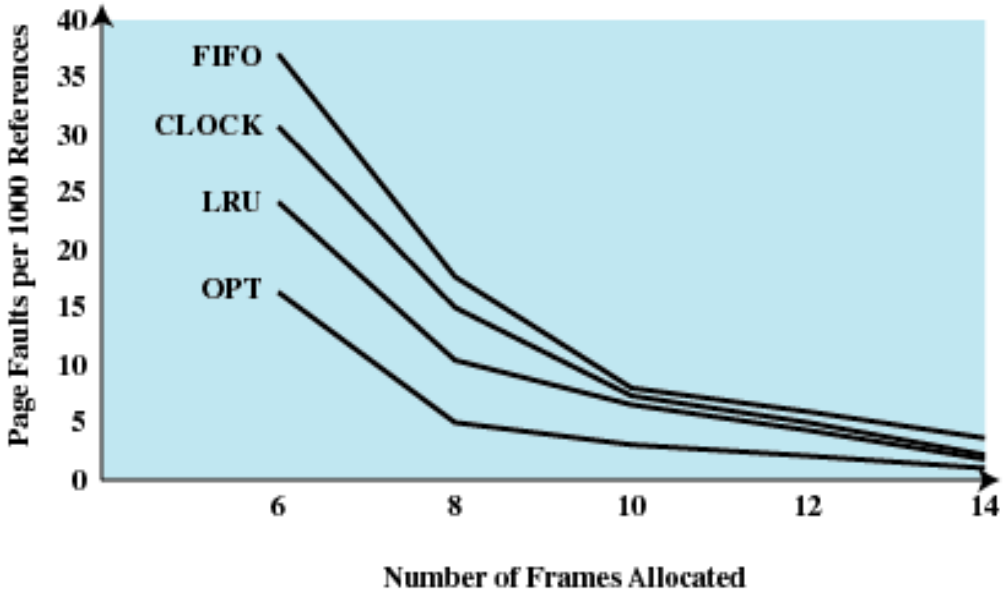


Figure 8.17 Comparison of Fixed-Allocation, Local Page Replacement Algorithms

Clock Page-Replacement Algorithm

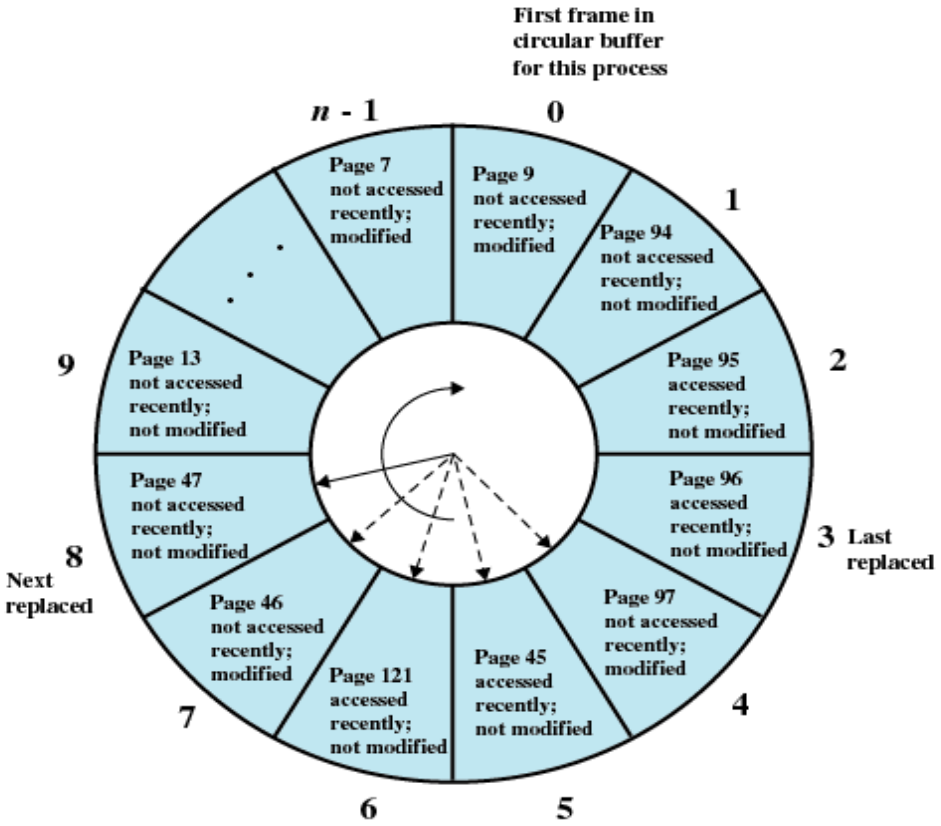


Figure 8.18 The Clock Page-Replacement Algorithm [GOLD89]

Page Buffering

- Replaced page is added to one of two lists
  - Free page list if page has not been modified
  - Modified page list

Resident Set Size

- Fixed-allocation
  - Gives a process a fixed number of pages within which to execute
  - When a page fault occurs, one of the pages of that process must be replaced
- Variable-allocation
  - Number of pages allocated to a process varies over the lifetime of the process

Fixed Allocation, Local Scope

- Decide ahead of time the amount of allocation to give a process
- If allocation is too small, there will be a high page fault rate
- If allocation is too large there will be too few programs in main memory

Variable Allocation,Global Scope

- Easiest to implement
- Adopted by many operating systems
- Operating system keeps list of free frames
- Free frame is added to resident set of process when a page fault occurs
- If no free frame, replaces one from another process

Variable Allocation,Local Scope

- When new process added, allocate number of page frames based on application type, program request, or other criteria
- When page fault occurs, select page from among the resident set of the process that suffers the fault
- Reevaluate allocation from time to time

Cleaning Policy

- Demand cleaning
  - A page is written out only when it has been selected for replacement
- Precleaning
  - Pages are written out in batches

Cleaning Policy

- Best approach uses page buffering
  - Replaced pages are placed in two lists
    - Modified and unmodified
  - Pages in the modified list are periodically written out in batches
  - Pages in the unmodified list are either reclaimed if referenced again or lost when its frame is assigned to another page

Load Control

- Determines the number of processes that will be resident in main memory
- Too few processes, many occasions when all processes will be blocked and much time will be spent in swapping
- Too many processes will lead to thrashing

Multiprogramming Effects

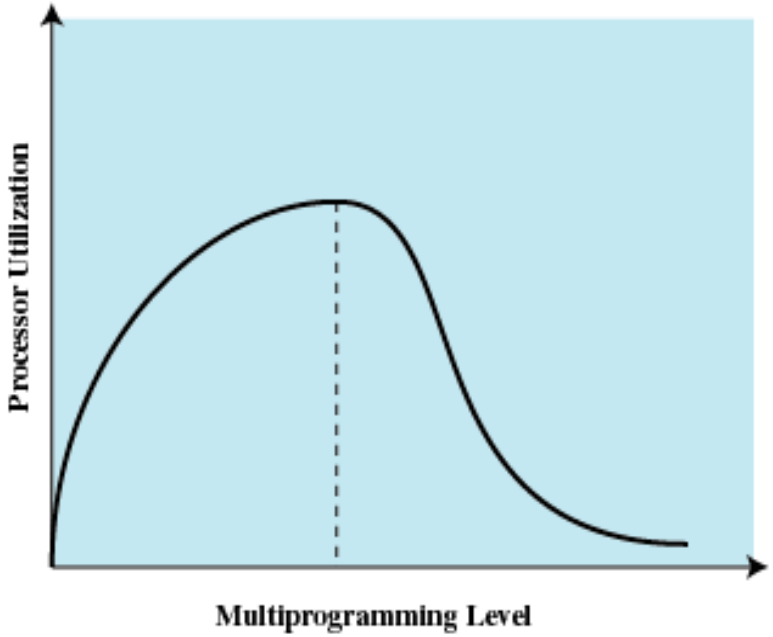


Figure 8.21 Multiprogramming Effects

Process Suspension

- Lowest priority process
- Faulting process
  - This process does not have its working set in main memory so it will be blocked anyway
- Last process activated
  - This process is least likely to have its working set resident
- Process with smallest resident set
  - This process requires the least future effort to reload
- Largest process
  - Obtains the most free frames
- Process with the largest remaining execution window

UNIX and Solaris Memory Management

- Paging System
  - Page table
  - Disk block descriptor
  - Page frame data table
  - Swap-use table

Table 8.5 UNIX SVR4 Memory Management Parameters (page 1 of 2)

Page Table Entry

Page frame number
Refers to frame in real memory.

Age
Indicates how long the page has been in memory without being referenced. The length and contents of this field are processor dependent.

Copy on write
Set when more than one process shares a page. If one of the processes writes into the page, a separate copy of the page must first be made for all other processes that share the page. This feature allows the copy operation to be deferred until necessary and avoided in cases where it turns out not to be necessary.

Modify
Indicates page has been modified.

Reference
Indicates page has been referenced. This bit is set to zero when the page is first loaded and may be periodically reset by the page replacement algorithm.

Valid
Indicates page is in main memory.

Protect
Indicates whether write operation is allowed.

Disk Block Descriptor

Swap device number
Logical device number of the secondary device that holds the corresponding page. This allows more than one device to be used for swapping.

Device block number
Block location of page on swap device.

Type of storage
Storage may be swap unit or executable file. In the latter case, there is an indication as to whether the virtual memory to be allocated should be cleared first.

Table 8.5 UNIX SVR4 Memory Management Parameters (page 2 of 2)

Page Frame Data Table Entry

Page State
Indicates whether this frame is available or has an associated page. In the latter case, the status of the page is specified: on swap device, in executable file, or DMA in progress.

Reference count
Number of processes that reference the page.

Logical device
Logical device that contains a copy of the page.

Block number
Block location of the page copy on the logical device.

Pfdata pointer
Pointer to other pfdata table entries on a list of free pages and on a hash queue of pages.

Swap-use Table Entry

Reference count
Number of page table entries that point to a page on the swap device.

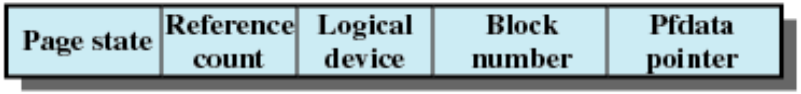
Page/storage unit number
Page identifier on storage unit.



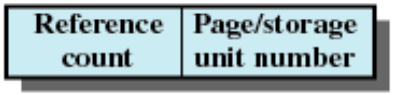
(a) Page table entry



(b) Disk block descriptor



(c) Page frame data table entry



(d) Swap-use table entry

Figure 8.22 UNIX SVR4 Memory Management Formats

UNIX and Solaris Memory Management

- Page Replacement
  - Refinement of the clock policy

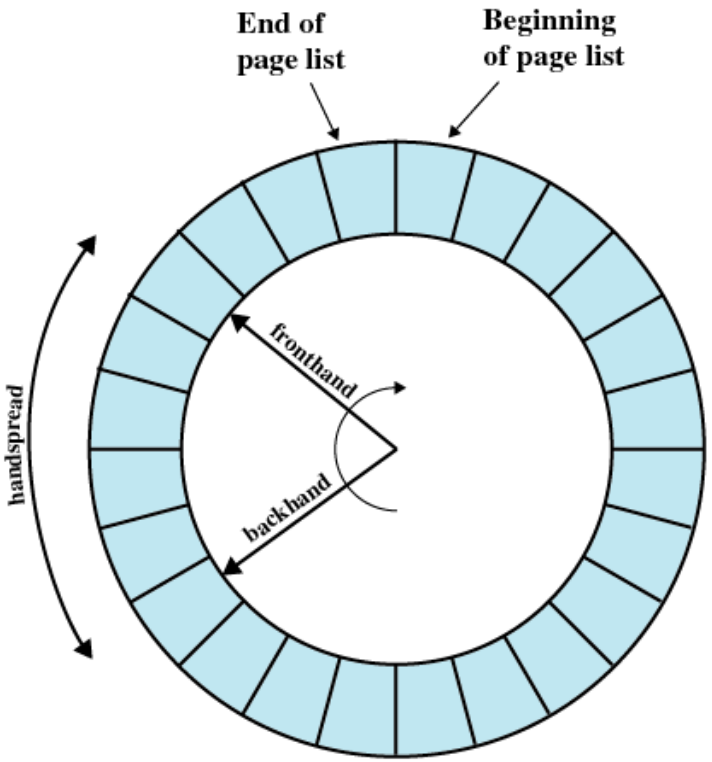


Figure 8.23 Two-Handed Clock Page-Replacement Algorithm

Kernel Memory Allocator

- Lazy buddy system

Initial value of D_i is 0
After an operation, the value of D_i is updated as follows

(I) if the next operation is a block allocate request:
  if there is any free block, select one to allocate
    if the selected block is locally free
      then $D_i := D_i + 2$
      else $D_i := D_i + 1$
  otherwise
    first get two blocks by splitting a larger one into two (recursive operation)
    allocate one and mark the other locally free
    D_i remains unchanged (but D may change for other block sizes because of the recursive call)

(II) if the next operation is a block free request
  Case $D_i \geq 2$
    mark it locally free and free it locally
    $D_i := D_i - 2$
  Case $D_i = 1$
    mark it globally free and free it globally; coalesce if possible
    $D_i := 0$
  Case $D_i = 0$
    mark it globally free and free it globally; coalesce if possible
    select one locally free block of size 2i and free it globally; coalesce if possible
    $D_i := 0$

Figure 8.24 Lazy Buddy System Algorithm

Linux Memory Management

- Page directory
- Page middle directory
- Page table

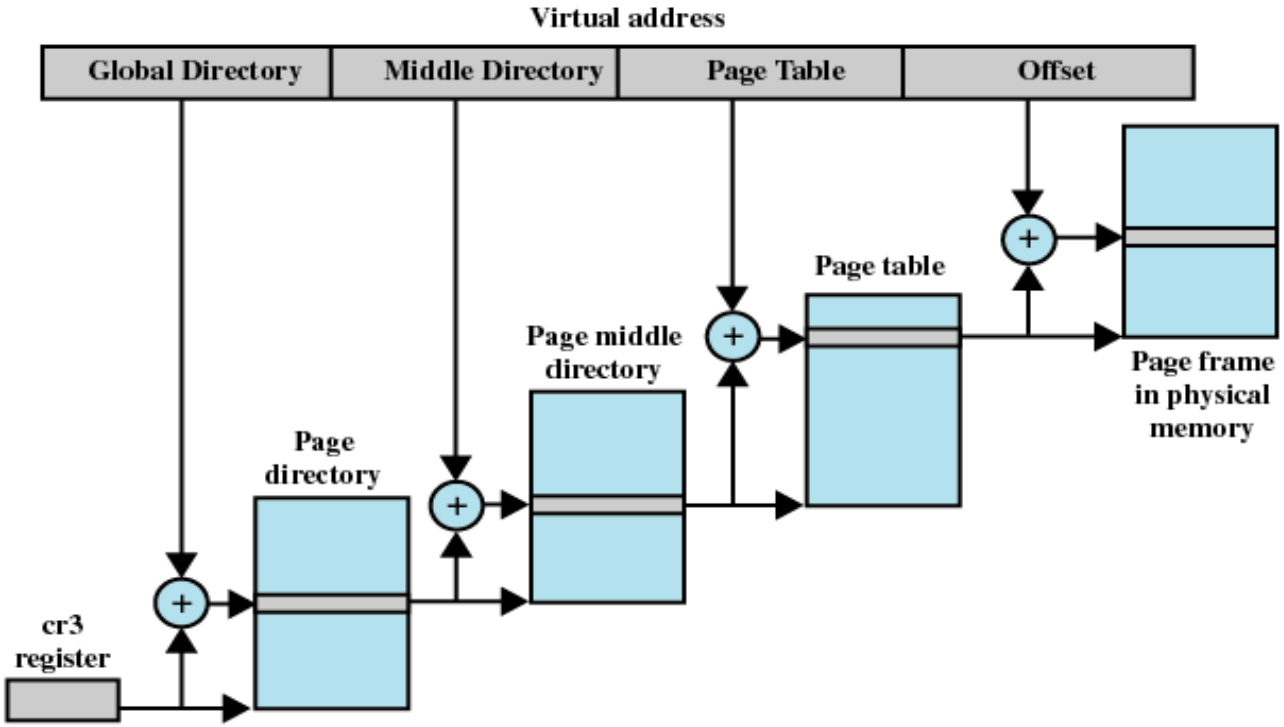


Figure 8.25 Address Translation in Linux Virtual Memory Scheme

Windows Virtual Address Space

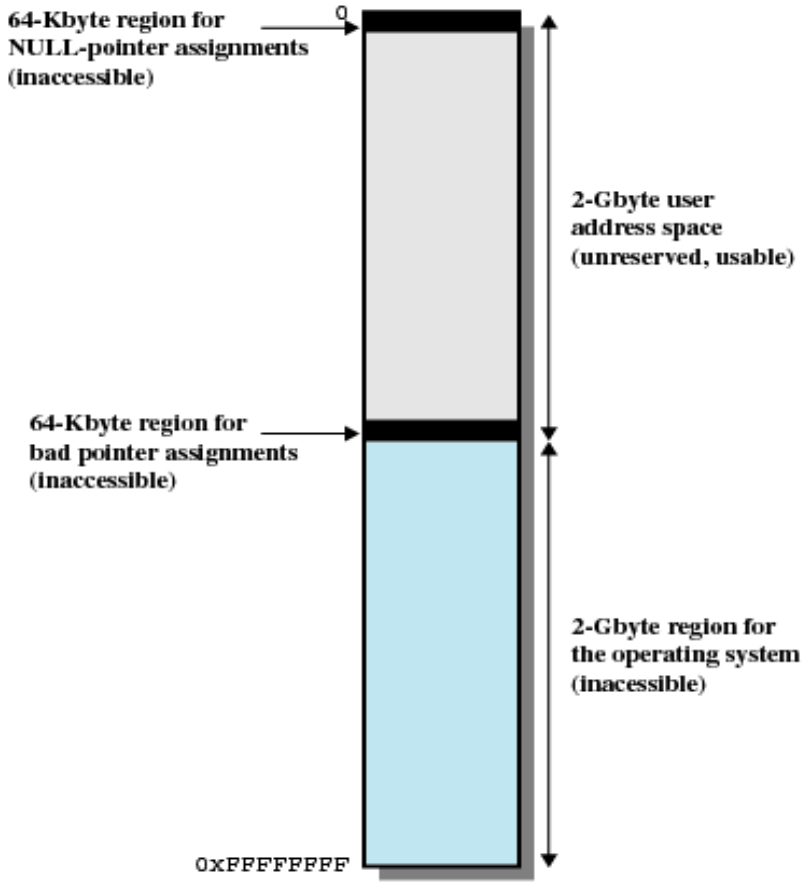


Figure 8.26 Windows Default Virtual Address Space

Windows Memory Management

- Paging
  - Available
  - Reserved
  - Committed